ZUNFT

To-Do for you

The things only you can do, as clear step-by-step guides: accounts to open, people to talk to, decisions to make. Each task says why it matters, what it costs, how long it takes and what to send me afterwards.

State: 11 October 2026 | Companion documents: **Zunft_Guide** and **Zunft_Tests**

Overview: what to do, and in what order

Nothing here blocks the testing on your own phones. These tasks decide when Zunft can safely go to **real customers** and when it can **charge money**.

	Task	Needed before	Cost	Time
☐	**1** Tax advisor meeting (questions are ready)	First real invoice	Advisor's fee	1 h meeting, 1-2 weeks to get one
☐	**2** Data protection basics (contracts with the AI/cloud providers, AI data settings)	Any outside tester with real data	Free	2-3 h
☐	**3** Lawyer review (privacy text, terms, working-time questions)	Selling to customers	Lawyer's fee	Weeks
☐	**4** Supabase for real use (paid plan, confirmation link, e-mail sender)	Any outside tester	Monthly	1-2 h
☐	**5** Signing key and Google Play	Installing on testers' phones the proper way	One-time fee	1 h, plus days of waiting for Google
☐	**6** Firebase for push messages	Nice to have	Free	30 min
☐	**7** Prices, plans and payments (Stripe)	Charging money	Per transaction	1 day to decide, 1 day to set up
☐	**8** Domain, e-mail addresses, Impressum	Public launch	Small	1-2 h
☐	**9** Cost alarms and key hygiene	Any outside tester	Free	30 min
☐	**10** The pilot with 2-3 real craftsmen	-	-	2-4 weeks
☐	**11** iPhone version (optional)	When you want iOS	Yearly fee	Needs a Mac
☐	**12** Name check and backups of your own	Any time	Free	1 h

SUGGESTED ORDER

Do **9** and **4** first (a day), then **2**, then book **1** and **3** because they take the longest to get an appointment for. While you wait: **5** and **6**. Run the tests from the Test Plan in parallel. Only after 1, 2, 3 and 4 give the app to outside craftsmen (task 10).

1. Tax advisor meeting

Why: the invoice texts and calculations are proposals until an advisor confirms them **Time:** 1 hour

Zunft prints a § 35a notice, splits labour and material, issues cancellations, sells material onward with a markup and now creates e-invoices. A wrong sentence on thousands of invoices is a real risk. The advisor's answers decide whether any wording or calculation changes.

PREPARE (30 MINUTES)

1. In the app, on your test business, create these examples and send them to yourself by e-mail as PDF: (a) an invoice for a **private** customer with labour, a flat fee and material; (b) an invoice for a **company** customer as an **e-invoice**; (c) a **Stornorechnung** of a test invoice; (d) a quote.
2. Export the receipt archive of one month: Belege → Einkaufsbelege → Exportieren (ZIP). Also share the monthly overview: Betrieb → Übersicht → Als Datei teilen (CSV).
3. Print **Zunft_Steuerberater.pdf** (in the docs folder). It lists about 20 questions with a box for the answer under each. Bring it, the printed examples and the ZIP on a USB stick or e-mailed.

AT THE MEETING

4. Start with the questions that can change the product most: **A1/A2** (flat fee as labour, the § 35a sentence), **C1/C2** (Stornorechnung and paid invoices), **E1/E2** (material bought for the customer, input VAT on till receipts), **E3** (keeping receipts, throwing away the paper), **W1** (e-invoice: ZUGFeRD enough or also XRechnung?), **W6** (billing tracked hours).
5. Ask two extra questions that are not on the list: (a) does the advisor want the export in a particular format (for example DATEV)? (b) does the business need a written **Verfahrensdokumentation** (GoBD process description), and may I draft it from the Project Guide?
6. Write the answers in the boxes. If the advisor says something is wrong, do not argue; just note the exact words.

AFTERWARDS

7. Photograph or type the answers and send them to me. I change the texts and calculations accordingly, add tests, and update the three documents.

> **GOOD TO KNOW**
>
> No answer is needed to keep testing. But do not send a real invoice to a real customer from Zunft before this meeting has happened.

2. Data protection basics (you can do this yourself)

Why: customer names, addresses and voices go to outside services | **Cost:** free | **Time:** 2-3 hours

Where the data goes: the **phone** → **Supabase** (database and files; your project is in the EU, Ireland) → **Deepgram** (voice to text) and **Google Gemini** (understanding text and receipt photos). The craftsmen who use Zunft are the *controllers* of their customers' data; you are their *processor*, and Deepgram, Google and Supabase are your *sub-processors*. You need a signed data-processing agreement (AVV / DPA) with each of the three, and the AI settings must be right.

2.1 SUPABASE

1. Open `supabase.com/dashboard`, click your organisation name (top left) → **Organization settings** → **Legal Documents** (also reachable through the "Legal" or "DPA" entry). Read the DPA and use the button to sign or request it.
2. Save the signed PDF in a folder called `Zunft-Vertraege` on your computer and in cloud storage.
3. Check the region once: project **zunft** → **Settings** → **General**: it must say a EU region (it does: eu-west-1, Ireland). Do not create the production project anywhere else.

2.2 GOOGLE GEMINI (THE MOST IMPORTANT SETTING)

1. Open `aistudio.google.com` → **API keys**. Find the key that Zunft uses and look at which **project** it belongs to.
2. Click **Set up billing** for that project (it opens Google Cloud). As far as I know, Google's terms treat content sent on the **free tier** differently from the **paid tier**: on the free tier Google may use the content to improve its products; on the paid tier it does not. Real customer data must only go through a project with billing enabled. **Please read Google's current "Gemini API Additional Terms" yourself**, because these terms change.
3. In the Google Cloud console for that project: **Billing** → **Budgets & alerts** → create a monthly budget (for example 20 EUR) with e-mail alerts at 50 %, 90 % and 100 %.
4. Find and accept the **Cloud Data Processing Addendum** (Google Cloud Console → **IAM & Admin** → **Privacy & Security** or the "Legal" page of Google Cloud). Save the confirmation.

2.3 DEEPGRAM

1. Open `console.deepgram.com`. Look in **Settings** / **Project** for any option about "model improvement" or data retention. Make sure you are **not** taking part in a programme that lets Deepgram keep or train on your audio. If you cannot find it, write to their support and ask in plain words: "Do you store or use our audio for training? Please send your DPA and your list of sub-processors."
2. Ask for the signed DPA and save it with the others. Ask where the audio is processed (several providers are in the USA; that needs a legal basis for international transfers, which the lawyer in task 3 checks).

2.4 AFTERWARDS

5. Send me: "DPA signed with Supabase / Google / Deepgram: yes or no, and where the audio is processed". I put the right names into the privacy text and the list of service providers for the lawyer.

UNTIL THIS IS DONE

Use only invented data (like in the Test Plan) and your own voice. Do not record real customers' names or addresses.

3. Lawyer review

Why: privacy text, terms and working-time questions are legal texts, not software

Time: weeks (appointment first)

Look for a lawyer who handles **IT law and data protection** (Fachanwalt für IT-Recht), ideally someone who has done software start-ups. Ask for a **fixed price** for the package below before you start.

3.1 WHAT TO BRING

- **Zunft_Guide.pdf** (what the app does) and the privacy screen of the app: Betrieb → Datenschutz (this is the draft employee notice).
- The list of service providers: Supabase (EU), Deepgram, Google Gemini (see task 2) and what each receives.
- A list of what is stored about people: employees (name, e-mail, role, assignments, notes with voice, appointments, working time), customers of the craftsmen (name, address, phone, e-mail), receipts (photos), recordings (deleted 90 days after the job closes by default).

3.2 WHAT TO ASK FOR

#	Task for the lawyer
3.1	**Privacy policy** for the app and the website, and the finished **employee notice** (the app shows it before someone joins a business)
3.2	**Contract with your customers** (the craftsmen): terms of use, and the **data-processing agreement (AVV)** in which you are the processor and the craftsman the controller; list of sub-processors
3.3	**International transfers:** is the use of Deepgram and Google (USA) covered (standard contractual clauses, EU-US framework)? Is audio of customers and employees acceptable? Is it enough to delete audio 90 days after a job closes?
3.4	**Liability wording:** the AI only proposes; the craftsman is responsible for the content, VAT rate and amounts of the invoice. How to say that in the terms and in the app
3.5	**Working time (Zeiterfassung):** (a) does the tracked time count as a working-time record in the legal sense, and what must a record contain? (b) **may the boss enter or correct time for a worker**? what must the worker be able to see and object to (the app marks such entries "vom Betrieb eingetragen" and the worker sees them in "Meine Zeiten"; there is no change log yet)? (c) is a works council's consent needed? (d) retention periods for time data
3.6	**Account closing:** an owner cannot delete the account in the app because invoices must be kept; the support process for closing a business, and what is deleted when
3.7	**Impressum** and price information rules for the website and the stores
3.8	**Customer data of the craftsmen's customers** in the calendar and the maps link ("Route" opens an address in Google Maps), and the confirmation texts

3.3 AFTERWARDS

1. Send me the finished privacy texts. I put them into the app (replacing the draft) and into the store listing material.
2. Tell me any rule that changes behaviour (for example "the worker must confirm entries" or "keep time data N years"). I build it and test it.

4. Supabase for real use

Why: the free plan pauses a project after a week without use | **Cost:** monthly plan | **Time:** 1-2 hours

> **EVIDENCE**
>
> Two of your older projects ("UniHub" and "unihub") show the status **INACTIVE**: they were paused. The same would happen to Zunft, and an app whose database is asleep does not work.

4.1 PAID PLAN

1. Dashboard → project **zunft** → **Settings** → **Billing** → **Change plan** → **Pro**. Enter a payment method.
2. Keep the **Spend cap** switched on (so a bug cannot create a huge bill). Open **Database** → **Backups** and check that daily backups are listed.

4.2 THE CONFIRMATION LINK (CURRENTLY LEADS TO AN ERROR PAGE)

When someone registers, Supabase e-mails a link. Today that link points to a page that does not exist. The simplest fix is a tiny web page that says "E-mail confirmed, please open the Zunft app."

3. On `github.com` create a new public repository called `zunft-site`. Add a file `index.html` containing a short sentence (for example: "Danke, deine E-Mail-Adresse ist bestätigt. Du kannst jetzt die Zunft-App öffnen und dich anmelden."). Commit.
4. In the repository: **Settings** → **Pages** → Source: **Deploy from a branch**, branch **main**, folder **/ (root)** → Save. After a minute GitHub shows the address (like `https://rb3111-ship.github.io/zunft-site/`).
5. Supabase → **Authentication** → **URL Configuration**: set **Site URL** to that address and add it under **Redirect URLs**. Save.
6. Authentication → **Sign In / Providers** → **Email**: switch **Confirm email** back **on** (you switched it off for testing). Register a test account and click the link: you must land on your page.

4.3 E-MAIL SENDER

Supabase's built-in e-mail service sends only a handful of mails per hour and is for testing. For real users you need your own sender (SMTP).

7. Create an account at an e-mail sending service (Resend, Brevo or Mailgun have free starter plans). Verify your domain there (task 8 gives you the domain; they show a few DNS entries to copy into your domain settings).
8. Supabase → **Authentication** → **Emails** → **SMTP Settings** → switch on custom SMTP and paste host, port, user, password and a sender address such as `noreply@yourdomain`. Also edit the German texts of the confirmation and password-reset e-mails (Authentication → Emails → Templates).

4.4 AFTERWARDS

9. Tell me when the plan is upgraded and the Site URL is set. Nothing in the app changes. I will then re-check that registration, the confirmation link and the password reset work from a phone.

5. Signing key and Google Play

Why: today the APK is signed with a debug key, which Google Play does not accept **Cost:** one-time Google fee

Time: 1 hour + days of waiting

> **CRITICAL: THE KEY IS IRREPLACEABLE**
>
> The upload key identifies your app. If you lose it or its passwords, updating the app becomes a long support case. Make two copies in different places (for example an encrypted USB stick and a password manager). **Never** put it in Git or send it by e-mail.

5.1 CREATE THE KEY

1. Open a terminal (PowerShell) and run (all on one line; it asks for two passwords and some questions about your name; use real values):

   ```
   D:\Android\jdk17\bin\keytool.exe -genkey -v -keystore D:\secrets\zunft-upload.jks -keyalg RSA -keysize 2048 -validity 10000 -alias zunft-upload
   ```

   Create the folder `D:\secrets` first. Write down: the file path, the store password, the key password, and the alias `zunft-upload` in your password manager.

2. Tell me: "key created at D:\secrets\zunft-upload.jks". **Do not send me the passwords in the chat.** I will prepare the build so that it reads the passwords from a local file (`android/key.properties`, excluded from Git) which you create yourself with four lines (storePassword, keyPassword, keyAlias, storeFile). I give you the exact content when you tell me.

5.2 GOOGLE PLAY CONSOLE

3. `play.google.com/console` → create a developer account (one-time fee, identity verification can take several days). Choose **Organisation** if you have a registered business: new *personal* accounts currently have to run a closed test with at least 12 testers for 14 days before they may publish (check Google's current rule).

4. Create the app: name **Zunft**, language German, free app, accept the declarations.

5. Prepare these inputs (I can draft the texts): short description (80 characters), full description, 4-8 phone screenshots, icon 512x512, a **privacy policy URL** (from the lawyer, task 3), and the **Data safety** form. Facts for the Data safety form: the app collects name, e-mail, audio recordings, photos and files, appointments and working time; data is encrypted in transit; users can request deletion; data is shared with processors (Supabase, Deepgram, Google).

6. Testing track: **Testing** → **Internal testing** → create a release → upload the app bundle (I build it with `flutter build appbundle` once the key exists) → add the testers' Gmail addresses → send them the opt-in link. This is the proper way to give the app to pilot craftsmen: updates arrive automatically.

6. Firebase for push messages

Why: so a worker is told when the boss gives him or changes an appointment while his app is closed **Cost:** free

Time: 30 minutes

Today reminders work without Firebase (the phone plans them itself). What is missing is a message *from the boss's action to the worker's phone*. That needs Google's push service (Firebase Cloud Messaging, FCM). You create the project; I build everything else.

6.1 CREATE THE PROJECT

1. Open `console.firebase.google.com` and sign in with your Google account. Click **Create a project** (or **Add project**), name it **Zunft**. Google Analytics: switch it **off** (not needed). Click **Create project**, then **Continue**.
2. On the project page click the **Android** icon ("Add app"). **Android package name**: exactly `de.zunft.zunft_app`. App nickname: Zunft. Leave the SHA-1 field empty for now. Click **Register app**.
3. Click **Download google-services.json**. Save it as `D:\Project_Alpha_X\zunft_app\android\app\google-services.json`. Then click **Next** / **Continue** through the remaining wizard screens (the code changes it shows are my job) and finish with **Continue to console**.

6.2 THE KEY THE SERVER NEEDS

4. Click the gear icon next to "Project Overview" → **Project settings** → tab **Service accounts**. Click **Generate new private key** → **Generate key**. A JSON file downloads.
5. Move it to `D:\secrets\zunft-fcm.json`. This file is a **secret** like a password: never in Git, never in a chat.
6. In **Project settings** → tab **Cloud Messaging** check that **Firebase Cloud Messaging API (V1)** says **Enabled**. If it says Disabled, use the three-dot menu there and open it in Google Cloud to enable it.

6.3 TELL ME

7. Write: "Firebase is ready". I then set the secret on the server myself from the file, add the Firebase plugin to the app, store each phone's address in the database, send a push from the database when an appointment is created, changed or cancelled for a person, add a settings switch, write the tests, and update the three documents. After the next APK build you install it on both phones and run check N45 (I add it to the Test Plan).

> **IPHONE**
>
> Push on iOS additionally needs an Apple Developer account and a Mac (task 11): in Firebase you would upload an APNs key. Android works without that.

7. Prices, plans and payments

Why: seats and storage are set by hand in SQL today **Time:** a day to decide, a day to set up

7.1 DECIDE FIRST (WRITE THE ANSWERS DOWN)

Question	Your answer
Plans: for example **Solo** (1 person) and **Team** (up to 5 / 10 people)?	
Monthly price per plan, and extra price per additional person?	
Storage included (default today: 1 GB per business) and price for more?	
Free trial (how many days)? What happens after it?	
What does it cost you per active business (AI, storage, support)? Measure during the pilot before fixing the price.	

7.2 UNTIL PAYMENTS EXIST: SET LIMITS BY HAND

In Supabase → **SQL Editor** → **New query**, replace the business name and run:

```
select id, name, seat_limit, storage_limit_mb from businesses;
update businesses set seat_limit = 5, storage_limit_mb = 5120 where name = 'Elektro Testmann GmbH';
```

The first line lists businesses with their limits; the second changes the seats (people) and the storage in MB for one business. The app cannot change these values, only you can.

7.3 SET UP STRIPE (WHEN THE PRICES ARE DECIDED)

1. `stripe.com` → create an account with your business details and bank account for payouts. Switch on SEPA direct debit and card payments. Ask your tax advisor about VAT on your own invoices (Stripe Tax can help).
2. **Product catalog** → create one product per plan with a monthly price. Note the price IDs.
3. Tell me the plan names, limits and price IDs. I build: a "Plan" screen, Stripe Checkout and the customer portal through a small server function, and a webhook that sets `seat_limit` and `storage_limit_mb` automatically. You give me the Stripe **test** keys first; the live keys only at launch, set as server secrets by me from a file you create.

8. Domain, e-mail addresses and Impressum

Why: the stores, the privacy text and the e-mail sender all need a real address **Time:** 1-2 hours

1. Check that the name is free: search the German trademark register (`register.dpma.de`) and the web for "Zunft" in software and craft services; check with a lawyer if in doubt. Then register a domain (for example zunft.app or zunft-handwerk.de) at a registrar of your choice.
2. Set up addresses: `support@` (the privacy text and the owner-account closing process refer to a support contact), `datenschutz@` and `noreply@` (for the sender in task 4).
3. Put up a simple website with an **Impressum**, the **privacy policy** and a short product description. The lawyer (task 3) provides the texts. The URLs go into the Play Console listing and into the app.
4. Send me the domain and the support address. I put them into the app text where it says "support" and into the documents.

9. Cost alarms and key hygiene

Why: a bug or an abusive user must not be able to create a large bill **Time:** 30 minutes

1. **Google (Gemini):** budget and alerts as in task 2.2, step 3.
2. **Deepgram:** console → **Billing** → set a usage alert or a spending limit.
3. **Supabase:** keep the spend cap on (task 4.1).
4. Check usage **once a week** during the pilot: Google AI Studio (usage), Deepgram (usage), Supabase (Reports). Write the numbers down; they decide your prices.
5. **Where the secrets are:** the Deepgram, Gemini and cron secrets are stored only as Supabase function secrets; the app contains only the public project key. If you suspect a leak, create a new key at the provider and tell me; I set it with `npx supabase secrets set NAME=value` and redeploy. Never paste a secret key into a chat or a document.

10. The pilot with real craftsmen

Why: only real use shows what is missing **Time:** 2-4 weeks

NOT BEFORE

Tasks 2 (data protection basics), 3.1/3.2 (privacy text and processing agreement at least as a draft), 4 (paid Supabase plan, confirmation link) and 9 (cost alarms). Real customers' data must not go through the app earlier.

1. Pick 2-3 craftsmen you trust: one working alone, one with a helper, ideally one who bills companies (property managers).
2. Give them a written agreement (the lawyer's draft): testing phase, no guarantee, data processing agreement, how to end it and get their data exported.
3. Install through the Play Console internal test track (task 5), or send the APK to the first one in person and sit with them for the first hour.
4. Ask them for one thing weekly: "Which invoice did you correct by hand or not send, and why?" Collect screenshots of every message that confused them.
5. Send me the collected findings. I turn them into fixes, in this order: wrong amounts, lost data, confusing messages, wishes.

11. iPhone version (optional)

Why: many craftsmen use iPhones **Cost:** Apple fee per year **Needs:** a Mac

iPhone apps can only be built on a Mac (or a cloud Mac service). The code is already written for both systems; what is missing is the build and Apple's paperwork.

1. Join the **Apple Developer Program** at `developer.apple.com/programs` (identity check; organisations need a D-U-N-S number, which takes extra time).
2. Get Mac access: your own Mac, a rented cloud Mac (for example MacinCloud) or a cloud build service for Flutter (for example Codemagic). Ask me which one fits; I prepare the project for it.
3. In **App Store Connect** create the app with the bundle ID I give you; upload via TestFlight to test with a few iPhones.
4. For push on iPhone: create an APNs key (Certificates, Identifiers & Profiles → Keys), upload it in Firebase → Project settings → Cloud Messaging → Apple app configuration (needs task 6).
5. Expect to answer Apple's review questions about privacy (same facts as the Play Data safety form) and to provide a demo account.

12. Name check and backups of your own

Time: 1 hour

1. **Code:** it is on GitHub (`Rb3111-ship/Zunft`). Check on `github.com` that the latest commit is there. Use a strong password and two-factor sign-in on that account.
2. **Keystore and secrets:** two copies of `zunft-upload.jks` and of `zunft-fcm.json` , plus the passwords, in different places (task 5).
3. **Database:** with the Pro plan Supabase makes daily backups (task 4.1). Once a quarter, ask me to test a restore on a scratch project so that you know it works.
4. **These documents:** keep the three PDFs and the signed contracts in one folder (`Zunft-Vertraege` for contracts).
5. **Accounts:** note which e-mail owns Supabase, Google Cloud / Firebase, Deepgram, GitHub, Stripe, Apple and Google Play, so that you are never locked out. Use one owner e-mail you control, plus a recovery option.

Sending me the results

For each task, send a short message like: "Task 4 done: Pro plan, Site URL set to ..., e-mail sender noreply@..., confirmation link works." Where the task says "do not send passwords", send only paths and names. I update the three documents and the working notes after each finished task.